

ZKOE 公司 Agents 开发交付要求

2025/04/12: v1; 2025/04/26: v2; 2025/09/25: v3; 2025/10/12: v4

V4 changes: 1. Add requirements for copyright and other annotations in source code. 2. Add options for saving agent data files. 3. Add new data type(matrix) for agent ports to aggregate multiple waveform data vectors. 4. Add specs for how to visualize(plot) the simulation results using MATLAB. 5. Require adding the date and version number to the folder naming.

V3 changes: 1. Change of Class Design with Structures for Agent Parameters and I/O Signals e.g., Structure of Parameters and Signals, respectively; 2. Interface with Middleware, as proposed by Qian Zhang.

V2 changes: 1. Addition of Class, Obj, variable naming conventions; 2. Specs of Construct method, contributed by Jingxin Ma, Wenbo Zhao, and Xubo Sun, as requested by Vincent Zhang.

V1: Documents developed by Qian Zhang, and modified by Jinxin Ma and shared with Zheda Group, as requested by Vincent Zhang.

一、 Agents开发交付内容

1. 程序源代码

(1) 需提供可运行的程序和程序完整源代码，且要在文件中标明版权和所有权声明 (Copyright Notice)，文件头注释 (File Header Comment)，修改历史 (Revision History)，模板如附录 1 所示。

(2) 命名规范要求：类的定义使用大驼峰命名，如 ClassTest。structure 的定义使用大驼峰命名。Agents 类的参数结构体统一命名为 Parameters，信号数据的结构体定义为 Signals。结构内部，例如 Signals 内部的信号向量应根据信号的种类和输入/输出端口使用小驼峰进行命名，如输入电信号为 eIn，再如输出光信号为 oOut1，oOut2…。变量和函数的定义使用小驼峰命名，如 variableTest；也可以使用特定缩写，如 square root 可命名为 sqrt。需要说明的是结构体的实例对象为变量，遵循小驼峰命名，例如 Laser::Parameters params 中的 params 属于变量，使用小驼峰命名。

(3) Agent 类名和文件名尽量一致。

(4) 每一个 Agent 代码交付包括对应名称的.cpp、.h 文件，并包含测试主文件.cpp，及编译后的 debug 文件包（包含测试数据及结果）。Agent cpp 包含而且仅包含一个 class。如果有继承关系，基类和派生类分别定义在不同的.cpp 和.h 文档。例如：

Laser.h, Laser.cpp, LaserTest.cpp

LaserDML.h, LaserDML.cpp, LaserDMLTest.cpp (注: LaserDML类继承Laser类)

2. Agent程序及接口说明

(1) Agent 程序说明

在本设计中, 每个 Agent 对应一个 C++ 类。类本身不显式定义构造函数和析构函数, 其功能由静态成员函数实现。每个类关联两个结构体, 并通过引用传递:

- 1) 参数结构体: 用于传递该 Agent 所需的属性、配置参数;
- 2) 信号数据结构体: 用于传递该 Agent 所需输入和输出的信号数据。

示例如下:

```
class FiberScalar {  
public:  
    struct Parameters {  
        double alpha;    // Attenuation (dB/m)  
        double beta2;    // Dispersion (s2/m)  
        double gamma;    // Nonlinearity (1/W/m)  
        std::string saveFile;  
    };  
    struct Signals{  
        cvec oIn;  
        cvec oOut;  
    };  
  
    static void execute(const Parameters& params, Signals& signals);  
};
```

如上面程序所示, Agent FiberScalar被定义为类 FiberScalar, 并关联两个结构体: Parameters 和 Signals。在函数调用过程中, 分别以引用的方式传递参数和信号数据, 保证在成员函数内部可以直接访问并修改所传原对象的数据, 当无需修改原数据时, 应使用const常量引用传递以确保高效、安全的数据传递。要求Agent的输出数据可以保存为文件, 输出文件的保存行为由Parameters结构体中的成员变量saveFile定义, saveFile的默认值设为"noSave", 当具有多个输出端口时应设置相应个数的成员变量, 如saveFile1, saveFile2...。具体规范如下:

1) 保存开关: 当saveFile的值为字符串"noSave"时, 程序不执行任何文件输出操作。否则程序执行文件输出操作, 输入字符串即为用户指定的输出文件名, 全称为字符串+".dat"。

2) 文件与格式: 文件保存时, 数据将写入用户指定的文件。该文件为纯文本格式(.txt), 仅包含数据本身, 不包含列标题、变量名或其他描述性信息。

3) 数据格式：基本规则：所有数据均以行向量的形式保存；复数处理：每个复数元素必须拆解为连续的实部与虚部。例如，复数行向量 $[a+bj \quad c+dj]$ 应被存储为 $[a \ b \ c \ d]$ ；矩阵处理：对于一个 $n \times m$ 的矩阵，应将其按行展开，存储为 n 行数据，每行（代表原矩阵的一个行向量）包含 m 个数据元素（注：复数时每行 $2m$ 个数据元素，所以matrix size为 $n \times 2m$ ）。为确保输出文件的可读性与可管理性，所有测试过程中保存的数据文件均应采用大驼峰命名法。文件名称应简洁、准确地反映所保存数据的物理意义，避免使用泛泛的或含义不明的词汇。示例：文件名可以为CWLaserOutput.dat（连续激光器输出）ModulatedSignalIQ.dat（调制后IQ信号数据）。

程序中 `execute` 方法被定义为静态、无返回值的静态成员函数，是仿真过程中 Agent 唯一直接执行的函数，实现 Agent 的主要功能。需要强调的是 `execute` 函数名即“execute”是软件部规定的不能修改的，而且函数中的形参如“params”等也推荐不要修改的。所以开发时可直接拷贝 `static void execute(const Parameters& params, Signals& signals)`放入 Agent 源程序。

在需要的情况下，例如 Agent 行为较为复杂，可用 divide and conquer 分解执行为多个方法（辅助函数），这时可在头文件中声明这些方法，再由 `execute` 调用以完成 Agent 具体行为操作。需要注意的是辅助函数也需要定义为静态函数，这是因为静态方法 (`execute`)不能直接调用非静态（普通类成员函数）方法。此外不同静态成员函数传递信息也应使用引用的方式，即不使用 `return`。具体完整的伪代码示例在文档的附录 2 所示。

上面示例中的 FiberScalar 类显示，我们使用 Agent 的 I/O 端口以及传递信号的种类来定义信号，而对于多输入多输出端口的情形，需使用多个端口变量，如下面程序中的 `cvec oIn1` 与 `cvec oIn2` 所示：

```
class Mux2 {
public:
    struct Parameters {
        std::string saveFile;
    };
    struct Signals {
        cvec oIn1;
        cvec oIn2;
        cvec oOut;
    };
    static void execute(const Parameters& params, Signals& signals);
};
```

此外，对于同一个 Agent 端口但有多组数据的情况，仍定义为一个端口变量，所以端口变量数目和 Agent 端口数目一致。例如 FiberVector 的输入端口 oIn 包含 X/Y 偏振方向的两组数据，仍然只能有一个变量 oIn，而数据建模为两行的一个矩阵，即 $\text{oIn}=[\text{oInX};\text{oInY}]$ ，其中 oInX 表示 X-pol 的复光场（行向量），oInY 表示 Y-pol 的复光场（行向量），oIn 是一个两行的矩阵，具体示例如下：

```
class FiberVector {
public:
    struct Parameters {
        double alpha;    // Attenuation (dB/m)
        double beta2;    // Dispersion (s^2/m)
        double gamma;    // Nonlinearity (1/W/m)
        std::string saveFile;
    };
    struct Signals {
        cmat oIn;
        cmat oOut;
    };

    static void execute(const Parameters& params, Signals& signals);
};
```

对仅有输出信号数据，无输入信号数据的类给出如下示例：

```
class Laser {
public:
    struct Parameters {
        double lamda;    // Laser wavelength [m]
        double P;        // Optical output power [dBm]
        double phase;    // Initial phase offset [rad]
        double LW;       // Laser linewidth [Hz]
        double Npoints;  // Number of sampling points
        double deltaT;   // Sampling time interval [s]
        double cLight;   // Speed of light in vacuum [m/s]
        double F0;       // Frequency offset [Hz]
        std::string saveFile;
    };
    struct Signals {
        cvec oOut;        // output light [w^(1/2)]
    };
    static void execute(const Parameters& params, Signals& signals);
};
```

即信号数据结构体 Signals 仅包含 Laser 的输出光。

对于派生类给出如下示例：

```
class LaserDML :public Laser{
public:
    struct Parameters:Laser::Parameters{
        double thresholdCurrent;    // Laser threshold current [mA]
        double slopeEfficiency;     // Laser slope efficiency [mW/mA]
        double maxCurrent;          // Maximum operating current [mA]
        double alpha;               // Linewidth enhancement factor (unitless)
        double kappa;               // Chirp coefficient [Hz/mW]
    };
    struct Signals {
        vec eIn;                    // Input drive signal vector (normalized 0-1)
        cvec oOut;                  // output light [ $w^{(1/2)}$ ]
    };
    static void execute(const Parameters& params, Signals& signals);
};
```

即 LaserDML 为基类 Laser 的派生类，且其参数结构体 Parameters 继承 Laser 的参数结构体 Parameters。这里需注意变量命名：派生类会继承基类中的物理量参数，这里同一个物理参数在基类和子类中能够区分，例如由“Laser::Parameters laserPara”声明的结构体 laserPara 的数据成员 laserPara.alpha 和由“LaserDML::Parameters laserDMLPara”声明的结构体 laserDMLPara 的数据成员 laserDMLPara.alpha，为不同对象的不同内存空间。

此外，Agent 类的功能描述，要写入程序注释，描述格式如下：

Agent 类的功能描述：.....

Agent 类具体示例执行后的输出：.....

示例：

Laser 类的功能描述：实现激光器输出信号的模拟。根据输入的 Parameters（如波长、输出功率、采样点数、相位、线宽等），生成对应长度的复数向量 signals.oOut。支持初始化输出光场的幅值和相位，可用于后续光信号仿真或调制。

Laser 示例执行后的输出：根据所输入的 Parameters，生成长度为 Npoints 的复数光场向量 signals.oOut。

（2）该Agent类中，静态成员函数说明，写入程序注释，描述格式如下：

静态成员函数名称：.....

静态成员函数的输入：.....

静态成员函数的输出：.....

静态成员函数实现的功能描述：.....

示例：

静态成员函数名称: static void execute(const Parameters& params, Signals& signals);

静态成员函数的输入: params // 类型 Laser::Parameters, 包含激光器的参数 (波长 lamda、功率 P、相位 phase、采样点数 Npoints、线宽 LW、采样间隔 deltaT、光速 cLight、频率偏移 FO 等);

静态成员函数的输出: 无返回值 void, 输出 laser 信号储存在 signals.oOut 中

静态成员函数实现的功能描述: 根据输入参数 params, 生成长度为 params.Npoints 的复数光场向量 signals.oOut。

3. 程序运行测试文档

每个 Agent 业务逻辑测试不少于 5 个测试用例, 需附测试用例具体参数, 运行后的结果截图如下:

```
=====
Test of Lasers
=====
LaserParameter
lamda = 1.55e-06 [m]
P = 10 [dBm]
phase = 0 [rad]
LW = 100000 [Hz]
Npoints = 50 [samples]
deltaT = 6.25e-12 [s]
cLight = 2.9979e+08 [m/s]
FO = 5e+07 [Hz]
=====
LaserSignal
Output light length = 50
Output light = [0.09999999-0.000143061i 0.0485567-0.08742i -0.052713-0.0849785i -0.0998945+0.00459194i -0.044804+0.0894013i 0.056304+0.082643i
0.0995961-0.00897849i 0.0407065-0.0913399i -0.0600323-0.0799758i -0.0990985+0.0133975i -0.0365797+0.0930695i 0.0634772+0.07727i 0.0984188-0.0177128i
0.0325706-0.0945471i -0.0667332-0.074476i -0.0975627+0.0219435i -0.0282678+0.0959215i 0.0700116+0.0714029i 0.0964548-0.0263907i 0.0239026-0.0971013i
-0.0731289-0.0682068i -0.0951954+0.0306241i -0.0196061+0.0980592i 0.076148+0.0648189i 0.09371-0.0349062i 0.0153574-0.0988137i -0.0789189-0.0614151i -
0.0920631+0.0390434i -0.0107244+0.0994233i 0.0816417+0.0577463i 0.0902295-0.0431119i 0.00624663-0.0998047i -0.0840967-0.0541087i -0.0882129+0.0471008
i -0.00183242+0.099832i 0.0863614+0.0504153i 0.0860904-0.0508632i -0.0025798-0.0999667i -0.0885674-0.0464308i -0.0837331+0.0546696i 0.00704484+0.099
7515i 0.090545+0.0424452i 0.0811653-0.058414i -0.0115542-0.0993303i -0.0924015-0.0382357i -0.0784349+0.062032i 0.0159924+0.0987129i 0.0939873+0.03415
23i 0.0756564-0.0653919i -0.0253324-0.0967381i] [W^(1/2)]
```

4. 运行结果可视化要求

如附录 3 MATLAB 代码模板所示, 我们要求对输入输出数据做充分的可视化, 命名与测试文件相同, 例如测试文件为 LaserTest.cpp, 则可视化文件为 LaserTest.m, 可视化的图片放置在程序运行测试文档中。

基本原则: 所有输入与输出数据均应进行充分的可视化分析, 从而验证系统性能并辅助问题诊断。

具体要求如下:

1) 输入数据可视化: 应对原始输入信号进行时域、频域或调制域的可视化, 建立性能分析的基准参考。

2) 输出数据处理原则: 输出数据的可视化必须从保存的数据文件中读取, 确保可视化结果与最终保存的数据完全一致, 保证分析结果的可重复性与真实性。

3) 对比分析: 在可能的情况下, 应将输入与输出数据置于同一视图或并列视图中进行对比, 直观展示系统处理 (如滤波、放大、编码等) 带来的影响。

4) 图表标注：所有图表应具备清晰的标题、轴标签、图例以及必要的性能指标（如 EVM、BER、SNR 等）标注。

5. 程序原理说明文档

此文档应包含该 Agent 实现的理论依据，可以为研发人员的详细公式推导（PPT，Word，Excel，matlab 程序等）、论文或者其他公开的权威资料。

二、 Agents开发交付形式

1. 文件夹命名格式，采用Agent实际名称命名，加注关键信息，如：Laser20251003v1p0，20251003为日期，v1p0代表version 1.0。
2. 文件命名格式，主要包括，样例所示：Laser.cpp、Laser.h、LaserTest.cpp(Laser测试的主程序文件，即main函数文件)，debug文件夹Laser_tests.docx（程序运行测试文档），CWLaserOutput.dat（输出数据的文本保存文件，即.txt文件），LaserTest.m（可视化），程序原理说明文档。

附录 1:

```

/*****
 * Copyright (c) 2025 Shaoxing Zhike Electro-Optics Computing Co. Ltd. All rights reserved.
 * Confidential and proprietary information.
 * Unauthorized copying, reproduction, or distribution of this software
 * is strictly prohibited.
 *****/

/**
 * @file: DummyEDFA.h
 * @brief:
 * @author:
 * @version: 1.1.0
 * @date: 2025-09-26
 * @department:
 * @project:
 */

/**
 * Revision History:
 *
 * Version      Date      Author      Changes
 * -----
 * 1.0.0        2025-09-18
 * 2.0.0        2025-09-26      Initial version
 *                Added EDFA noise modeling
 */
```

附录 2:

```
-----DummyEDFA.h-----  
  
//此处略去copyright和版本等信息  
//.....  
  
//include needed header files  
#include <string>  
#include <fstream>  
#include <iomanip>  
#include <iostream>  
#include <cmath>  
#include <vec.h>  
  
.....  
//using ZK namespace  
namespace ZK {  
  
    /*  
    * DummyEDFA class functionality:  
    *     Simulates the EDFA.  
    *     .....  
    * Output after executing a EDFA:  
    *     Generates a amplified optical signal with niose.  
    */  
    //define agent class  
    class DummyEDFA {  
    public:  
        struct Parameters {           //Agent Parameters Struct  
            double frequency;  
            double bandwidthScaling;  
            double rateSymbols;  
            double lineWidth;  
            double length;  
            .....  
            std::string saveFile1;  
            std::string saveFile2;  
        };  
        struct Signals {              //I/O Signals Struct  
            cvec oIn1;                 //optical signal in  
            cvec oIn2;                 //pump laser signal in  
            cvec oOut1;                //amplified optical signal out (with ASE noise added)  
            cvec oOut2;                //ASE noise out.  
            .....  
        };  
  
        // the following statement just copy paste: don' t change
```

```
static void execute(const Parameters& params, Signals& signals);

//the following statements are customized for this specific Agent class
//the following functions also need to be defined as static functions (class
functions).
//the following input gain parameter change its value by reference
static void calculateGain(const Parameters& params, cvec& pumpSignal, double& gain);
//the following ampSignal vec references to the input optical signal and then be used
as return vec of amplified optical signal after calculateAmpSignal function
static void calculateAmpSignal(double gain, cvec& ampSignal);
//the following noiseSamples vec references to a return vec
static void calculateNoise(const Parameters& params, const double gain, cvec&
noiseSamples);
static bool saveUsedData(const cvec& data, const std::string& saveFile);
.....
};
}
```

-----DummyEDFA. cpp-----

```
//此处略去copyright和版本等信息
//.....
```

```
#include "DummyEDFA.h"
//using ZK namespace
namespace ZK
{
    /**
    * Static member function name:
    * void DummyEDFA::execute(const Parameters& params, Signals& signals)
    *
    * Input:
    * - params (type: DummyEDFA::Parameters)
    * .....
    *
    * - signals (type: DummyEDFA::Signals)
    * Signals.oIn1 is the input optical signal and Signals.oIn2 is the input pump laser
    *
    * Output:
    * - void (no return value). and the amplified optical signal with noise is in
signals.oOut1
    * and the noise is in signals.oOut2
    *
    * Functionality:
    * amplifies the input signal signals.oIn1, and the noise is also added
```

```
*    according to the input parameters.
*/
void DummyEDFA::execute(const Parameters& params, Signals& signals)
{
    double G = 0;          //The initialization of the gain of EDFA and will be
modified
    cvec noiseSample;      //The noise vector to be calculated and cvec is automatic
initialized
    calculateGain(params, signals.oIn2, G); //Only a part of the params is used for
calculating G.
    calculateAmpSignal(G, signals.oIn1); //signals.oIn1 = signals.oIn1*G;...
    calculateNoise(params, G, noiseSample); //Only a part of the params is used noise
vector noiseSample
    signals.oOut1 = signals.oIn+noiseSample; //Get the amplified optical signal with ASE
noise using the amplified optical signal signals.oIn and the ASE noise noiseSample
    signals.oOut2 = noiseSample; //Get the ASE noise
    // Save amplified signal with noise if requested
    if(saveFile == "noSave") return;
    saveUsedData(signals.oOut1, params.saveFile1);
    saveUsedData(signals.oOut2, params.saveFile2);

}
/**
* Static member function name:
*    void DummyEDFA::calculateGain(const Parameters& params, cvec& pumpSignal, double& gain)
*
* Input:
*    - params    (type: DummyEDFA::Parameters)
*    .....
*
*    - pumpSignal (type: cvec)
*    the pump laser
*
* Output:
*    - void (no return value). and the output gain is in 'gain'
*
* Functionality:
*    uses a part of params and the pump laser to calculate the gain and place it in 'gain'
*/
//The special function that implements the agent's functionality, which is called by the
"execute" function.
void DummyEDFA::calculateGain(const Parameters& params, cvec& pumpSignal, double& gain);
{
    gain = params.... + ...; //Calculate the gain using the parameters in the agent params
and the pumpSignal.
```

```
    }  
    /**  
    * Static member function name:  
    *     void DummyEDFA::calculateAmpSignal(double gain, cvec& ampSignal)  
    *  
    * Input:  
    *     - params   (type: DummyEDFA::Parameters)  
    *       .....  
    *  
    *     - ampSignal (type: cvec)  
    *       the input signal(will be amplified)  
    *  
    * Output:  
    *     - void (no return value). and the amplified optical signal is in 'ampSignal'  
    *  
    * Functionality:  
    *     Multiply the gain with the input signal 'ampSignal', and place the amplified signal in  
    *     its original position 'ampSignal'  
    */  
    void DummyEDFA::calculateAmpSignal(double gain, cvec& ampSignal) ;  
    {  
        ampSignal = ampSignal * gain;//Amplify the input optical signal  
    }  
    /**  
    * Static member function name:  
    *     void DummyEDFA::calculateNoise(const Parameters& params, const double gain, cvec&  
    *     noiseSamples);  
    *  
    * Input:  
    *     - params   (type: DummyEDFA::Parameters)  
    *       .....  
    *  
    * Output:  
    *     - void (no return value). and the noise is in 'noiseSamples'  
    *  
    * Functionality:  
    *     create the noise according to a part of params  
    */  
    void DummyEDFA::calculateNoise(const Parameters& params, const double gain, cvec&  
    noiseSamples);  
    {  
        noiseSamples.set_size(params.length);  
        for(int i= 0;i<params.length;++i)  
        {  
            noiseSamples[i] = params....+...;//Calculate the noiseSamples using the parameters
```

```
in params
    }
}

bool saveUsedData(const cvec& data, const std::string& saveFile) {
    // Check if saving is disabled
    if (saveFile == "noSave" || saveFile.empty()) {
        return true; // Not an error if saving is disabled
    }

    // Validate input data
    if (data.empty()) {
        std::cerr << "Warning: Complex vector data is empty, skipping save operation for: "
            << saveFile << std::endl;
        return false;
    }

    // Open output file
    std::ofstream outFile(saveFile);
    if (!outFile.is_open()) {
        std::cerr << "Error: Cannot open file for writing: " << saveFile << std::endl;
        return false;
    }

    // Set output precision
    outFile << std::fixed << std::setprecision(6);

    // Save as row vector with consecutive real and imaginary parts
    for (size_t i = 0; i < data.size(); ++i) {
        outFile << data[i].real() << " " << data[i].imag();
        if (i < data.size() - 1) {
            outFile << " "; // Add space between numbers
        }
    }

    outFile.close();
    std::cout << "Complex vector data saved to: " << saveFile << std::endl;
    return true;
}
}
```

```
----- DummyEDFATest.cpp -----  
  
//此处略去copyright和版本等信息  
//.....  
  
#include "DummyEDFA.h"  
using namespace std;  
int main() {  
    //set agent parameters  
    double frequency = 193e12;  
    .....  
    string saveFile1= "AmplifiedSignalWithNoise.dat";  
    string saveFile2="EDFANoise.dat";  
    //define parameters structure of agent  
    ZK::DummyEDFA::Parameters dummyEDFAParams{ frequency,..... };  
    //define signal structure of agent  
    ZK::DummyEDFA::Signals dummyEDFASignals;  
    //set signal input vector  
    dummyEDFASignals.oIn1.set_size(params.Npoints);  
    dummyEDFASignals.oIn1 = [...];  
    dummyEDFASignals.oIn2.set_size(params.Npoints);  
    dummyEDFASignals.oIn2 = [...];  
  
    //execute the function  
    ZK::DummyEDFA::execute(dummyEDFAParams, dummySignals);  
  
    //Output simulation parameters and signals  
    cout << "===== " << endl;  
    cout << "                Test of DummyEDFA                " << endl;  
    cout << "===== " << endl;  
    cout << "                DummyEDFA Parameters                " << endl;  
    cout << "lamda    = " << dummyEDFAParams.frequency << " [Hz]" << endl;  
    .....  
    cout << "===== " << endl;  
    cout << "                DummyEDFA Signals                " << endl;  
    cout << "Output light length = " << dummySignals.oOut1.length() << endl;  
    cout << "Output light      = " << dummySignals.oOut1 << " [W^(1/2)]" << endl;  
    .....  
}
```

附录3:

```
-----LaserTest.m-----

%% General Graphic Settings
set(0, 'DefaultAxesFontSize', 12);
set(0, 'DefaultAxesLineWidth', 1.2);
set(0, 'DefaultLineLineWidth', 1.8);
set(0, 'DefaultFigureColor', 'white');

%% Read data from CWLaserOutput.dat and plot waveform
figure('Position', [100, 100, 1000, 700]);

% Read data from file
data = load('CWLaserOutput.dat'); % Load data file

% Parse data - adjust according to your data format
% Assuming data format: real1 imag1 real2 imag2 ...
if mod(length(data), 2) == 0
    % Data is complex: real and imaginary parts arranged alternately
    laser_complex = data(1:2:end) + 1j * data(2:2:end);
else
    % Data might be pure real or pure imaginary, adjust accordingly
    error('Data length is not even, please check data format');
end

% Create time vector
t = (0:length(laser_complex)-1) * 1e-10; % Assuming 100ps sampling interval

% Plot optical field intensity (power)
subplot(2, 1, 1);
optical_power = abs(laser_complex).^2;
plot(t*1e9, optical_power, 'r-', 'LineWidth', 2);
ylabel('Optical Power (W)');
title('Laser Field Intensity Envelope');
grid on;
xlim([0, max(t)*1e9]);

% Plot real part of optical field
subplot(2, 1, 2);
plot(t(1:min(1000, end))*1e9, real(laser_complex(1:min(1000, end))), 'b-', 'LineWidth', 1.5);
xlabel('Time (ns)');
ylabel('Electric Field Real Part (W^{1/2})');
title('Laser Electric Field Waveform (Initial Segment)');
grid on;

sgtitle('Testing: Laser Time-Domain Characteristics', 'FontSize', 14, 'FontWeight', 'bold');
```